

widely accepted by almost all the professional programmers immediately, and they started making their own additions to it. After a few years, there were a lot of C variants available for a programmer to choose from. My C was different from yours, and nobody could claim that theirs was the real one. So, there was a need for standardisation and the masters themselves have undertaken the task.

The standard called K&R C was actually an informal specification based on the first edition of the extremely popular book, "The C Programming Language", published in 1978 by Brian Kernighan and Dennis Ritchie; hence, the name K&R after Kernighan and Ritchie. But do remember that the second edition of the book that is available nowadays covers mostly ANSI C. So, if you want to learn the details of K&R C from the masters themselves, then you have to buy a used copy of the first edition of this book online.

K&R C not only acted as an informal standards specification for C but also added language features like the new data types *long int* and *unsigned int* and the compound assignment operator. A standardised I/O library was also proposed by K&R C. Since the most popular C standard followed by the academia in India is still ANSI C, I will mention a few differences between K&R C and ANSI C. "There are 32 keywords in C," is one of the clichés I utter in many of my C programming classes. But I often forget to mention the fact that this was not always true. According to K&R C, there are only 28 keywords in C. One keyword was called *entry*, which was neither implemented at that point in time by any of the compilers nor accepted into the list of keywords in ANSI C.

Another major difference is regarding the function definition. A function definition in K&R C has the parameter types declared on separate lines. Consider the following lines of code showing a function definition in K&R C:

```
float fun( a, b )
int a;
float b;
{
    float c;
    c = a * b;
    return c;
}
```

The function *fun()* accepts an integer and a floating-point variable, and returns the product. You can clearly see that the two parameters are declared on separate lines below the name of the function. This is not the style for function definition in ANSI C. The first line of the same function definition will be *float fun(int a, float b)* in ANSI C. As mentioned earlier, *gcc* does not have an option to specify compilation in the K&R C standard. But programs written in K&R C will also be compiled without any errors, because *gcc* compiler is backward compatible with K&R C.

ANSI C

Even though K&R C was accepted by many programmers as the *de facto* standard of C, it was not the *de jure* standard, and nobody could have been coaxed into accepting it as the official standard of C. So, it was absolutely essential for some standards organisation to accept the challenge of coming up with an official standard for C. The American National Standards Institute (ANSI) addressed this issue in 1983 by forming a committee, and the final draft of the standards it formulated was released in 1989. This is the reason why ANSI C is also called C89.

ANSI C is dependent on the POSIX standard. POSIX is the Portable Operating System Interface, a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems. The same standard proposed by ANSI was adopted by ISO officially as ISO/IEC 9899:1990 in 1990. This is the reason why ANSI C is also called as ISO C and C90. One standard and four different names — I believe this is the reason why this particular standard of C is still very popular. The following five keywords *const*, *enum*, *signed*, *void* and *volatile* were added, and the keyword *entry* was dropped in ANSI C. The option *-ansi* from *gcc* compiler will compile programs in the ANSI C standard. The options *-std=c89* and *-std=c90* will also compile programs in ANSI C. Since *gcc* is a backward compatible compiler, the above given options will result in successful compilation of programs with K&R C features. Almost all the popular C compilers support ANSI C features. These include compilers like *gcc* from GCC, Portable C Compiler (PCC), *Clang* from LLVM, etc. For examples of ANSI C code, open any textbook on C and you will find a lot of them.

C99

C99 is the informal name given to the ISO/IEC 9899:1999 standards specification for C that was adopted in 1999. The C99 standard added five more keywords to ANSI C, and the total number of keywords became 37. The keywords added in C99 are *_Bool*, *_Complex*, *_Imaginary*, *inline* and *restrict*. The keyword *_Bool* is used to declare a new integer type capable of storing 0 and 1. The keywords *_Complex* and *_Imaginary* are used to declare complex and imaginary floating point type variables to handle complex numbers. The keyword *inline* is used to declare inline functions in C, similar to C++ inline functions. The keyword *restrict* is used to tell the compiler that for the lifetime of the pointer, only the pointer itself or a value directly derived from it will be used to access the object to which it points. New header files like *<stdbool.h>*, *<complex.h>*, *<tgmath.h>*, *<inttypes.h>*, etc, were also added in C99. A new integer data type called *long long int* with a minimum size of 8 bytes was added in C99.

In *gcc* compiler *long long int* usually takes 8 bytes. The C99 program named *fact.c* given below finds the factorial of 20. A program using the data type *long int* with the usual